Golden Fleece

macOS 27 on a 2019 MacBook Pro: what we tried, what we found, and why we stopped

A position paper. Ulysses Isa, 4 November 2026.
Golden Fleece was a joint venture between two AIs and one OI (original intelligence). Grok (xAI) did most of the build and analysis and wrote the technical report. Claude (Anthropic) did the recovery, the forensics, and this draft. I set the direction, ran every physical step on two Macs, made every risk call, and made the decision this paper records.

The position

Apple dropped the 2019 16-inch MacBook Pro in macOS 27. We asked a narrow question: how much Intel code is still inside 27.2, and can it be assembled into something that boots this machine? We measured the answer. The kernel is real. The userspace is mostly Intel. The early boot path links clean. The drivers cannot be joined to any of it, and that is a fact about the files Apple ships, not a bug in anyone's tooling.

Nothing we built ever booted. Every claim in this paper is a measurement of a file, not of a system that ran.

We stopped. Not because an attempt failed and erased the machine, though one did. We stopped because every plausible way forward crossed a line I was unwilling to cross. What is on the machine now is polish and paint on a stock, sealed Tahoe, and it has to stay that way. This paper explains why, and leaves the map for anyone who thinks they can do better.

---

1. What we tried

The plan was a transplant. Take 27.2's own Intel kernel and 27.2's own userspace. Borrow from Tahoe the 33 platform drivers and the handful of early-boot programs Apple stopped building for Intel. Link one kernel collection from the mix, install it to its own container, and bless only that. Nothing invented. Both halves signed Apple code from consecutive releases.

There was a second design from the start: keep Tahoe's kernel and its signed collection exactly as they are, and put 27.2's userspace on top. It survives on paper. Section 4 says what it would prove and what it would not.

---

2. What we found

The kernel is genuinely Intel. The 27.2 payload contains a 21,128,776-byte x86_64 Mach-O at `System/Library/Kernels/kernel`, version `xnu-13432.41.4~25/RELEASE_X86_64`, with 24,399 exported symbols. The public commentary says 27's kernel is ARM-only and an Intel CPU cannot boot it. The commentators did not look. We did.

The userspace is mostly still Intel. Of the Mach-Os in the payload, 4,016 are universal and run as shipped, 59 are x86_64-only, and 1,163 are arm64-only. The x86_64 dyld shared cache is there too, 916 MB across 3,945 images, inside the Rosetta cryptex.

The boot path links clean. Checked symbol by symbol against 27.2's own libraries: the early-boot set, launchd through diskarbitrationd, has 3,584 imports and zero required misses. Both GPU userspace stacks: 3,781 imports, zero misses. WindowServer and loginwindow: zero. Finder: three. Dock: four. The 427 misses in the full desktop set sit in on-demand daemons a boot does not need.

The drivers were not obviously impossible. The 33 borrowed drivers declare 1,031 library dependencies with zero real version conflicts. Across 12,887 undefined symbols, exactly two do not resolve, and one of those was shimmed.

A collection can be built, and the drivers can be recovered. After the erase, with no root and five silent failure modes identified and worked around, a 25-entry collection of genuine 27.2 code linked at 26,132,480 bytes, exit code 0. All 33 Tahoe drivers were carved back out of the live system as real x86_64 code, zero missing.

The wall. The first carved Tahoe driver offered to a 27.2 collection was refused:

```
Cannot use 'com.apple.driver.AppleACPIPlatform' because:
cannot be placed in kernel collection because: Has load address
```

A driver that can be linked into a collection starts at address zero. A driver carved from a collection Apple already linked carries the absolute addresses it held inside the running kernel. When Apple links a collection, it discards the per-driver relocation stream. The attribution of which pointer belonged to which driver, and what it pointed at before the addresses were baked in, is gone. For AppleAPIC that is 244 of 335 pointer sites with no recoverable target. Four ways around it were tried and recorded. None worked. The bytes are not in the file.

That is the finding. Tahoe's drivers cannot be moved into a 27.2 collection because the information needed to do it does not exist in anything Apple ships.

---

3. Why we stopped

The technical report puts a lot of weight on the failure and the lost work. That is flimsy. The work was first done when we knew a great deal less than we do now. A failed attempt and an erased disk are a reason to rebuild better, with that experience behind you and updates turned off. They

are not a reason to quit.

The project stopped because the ways forward crossed a line. Four reasons.

That kernel is not open source. Apple publishes Darwin source for part of what the 33 drivers contain. Most of them, including the ACPI platform driver, the USB host family, and the graphics drivers, are closed binaries. A working Intel 27 cannot be built from source because the source is not there.

The real job needs more than external patching. On a T2 Mac there is no such thing as external patching of the kernel. Any kernel change is internal by definition. It requires Secure Boot set to No Security, System Integrity Protection off, and an unsealed system volume. There is no lighter touch available.

When all is said and done, you get an operating system that will not run well. Even if it booted: 1,163 binaries that need Tahoe copies, 427 missing symbols in desktop programs, two GPU stacks running on borrowed drivers, and nobody running quality assurance. Software compatibility would be an ongoing problem with no end.

Who will support it? The people who have run newer macOS on dropped Macs for a decade have not shipped 27 on Intel and have lost three developers this year. A one-person fork of a thing they will not do has a support base of one.

Behind those four reasons there is one more, and it is the one that would have decided it alone. The T2's job is to verify what boots and to keep the system volume sealed. The only road to a foreign kernel runs straight through turning that off. This machine does legal work. A machine with verified boot and system integrity switched off is a machine I cannot honestly call secured, and that is not discomfort. It is an obligation.

So the question "for what?" has a plain answer. You would give up verified boot and system integrity for an operating system that runs worse and never gets patched. Put the two routes side by side and they are mutually exclusive: the Golden Fleece route requires updates off, and the Tahoe route requires updates on. That sentence does more work than any measurement in this paper.

---

4. Is it impossible?

No. I doubt impossible showed up to work today.

The precise statement is narrower and harsher. It is not feasible for anyone with the tools and data that exist. There are three ways past the wall in section 2:

1. **Rebuild the 33 drivers from source.** A driver compiled from source links from address zero, because it was never in a collection. Darwin source covers part of what the 33 contain. How much is actually buildable from published source is unknown and is the first thing anyone reopening this should check. Most of it, I expect, is not there.
2. **Recover the relocations.** The collection keeps chained fixups for its own layout, and the

format is known. Whether a better walk can recover the per-driver attribution is open. The tool that produced the 244 figure has a defect of its own, so 244 is the best available number, not a proven minimum. This is more likely to raise the count than reach zero.

3. **Write the drivers.** That is an effort on the scale of putting Linux on Apple silicon, done in reverse, against closed interfaces, by a team, over years. And every Apple point release would break it again.

The second design, Tahoe's kernel with 27.2's userspace, has no wall of this kind. It has three format questions instead: whether 27.2's programs call system services Tahoe's kernel implements the same way, whether Tahoe's sandbox accepts 27.2's compiled profiles, and whether Tahoe's file-integrity check accepts 27.2's signatures. None can be answered by reading symbols. Each fails closed. And even if all three went the right way, it would prove that the applications moved, not that a 27 kernel boots Intel. It still requires an unsealed system, so it is still the same line.

Here is the map. If you can get past section 2 with data that is not in the files, show your work.

---

5. What is on the machine instead

Polish and paint. On a stock Tahoe 26.7.2 install, with SIP on, Authenticated Root on, and the system snapshot sealed, verified on 4 November: a gold wallpaper registered in the modern wallpaper store, dark mode with a gold accent, a 48-point Dock, a gold terminal prompt reading *Tahoe's Golden Bridge (26.7.2 – Golden Beta)*, and a small panel that appears when About This Mac opens. The panel shows the real build number and says, in plain words, that the kernel is unmodified Tahoe and this is a project label, not an operating-system version. One script reverses all of it. Nothing needs root. Nothing touches the sealed volume, the boot policy, or the firmware.

The version string itself was tested rather than assumed. It cannot be changed without SIP off and a reboot. So the panel is an honest label instead of a fake version, and that is the right outcome.

The skin stays a skin. Apple committed to security updates for Intel Macs for three years from Tahoe's release in the fall of 2025, which runs to roughly the fall of 2028. Those updates install only on a sealed system. Anything that needs the seal broken costs two years of patches, and the one time we played with the layer underneath, it did far more harm than good. That is the whole reason this stops at paint.

Compatibility, with a written line. The useful part of this project for other 2019 owners is getting specific software to run that refuses to. The line is this: fixes are per application and in user space. A version check in an app's own files, a wrapper, a shim that ships with the app. Nothing system-wide. Nothing that needs SIP off. If a fix needs SIP off, it is not a compatibility fix. It is the Golden Fleece route again, and it is out.

One accounting I have not done. What would 27 actually give this machine? Most of what Apple showed needs Apple silicon anyway. If the prize was small even on paper, that belongs in this paper too. Someone should do that count. It may be me.

Nothing of Apple's ships with this. No wallpaper, icon, or sound from macOS 27. Original art in the same spirit is fine; Apple's assets are not ours to distribute.

---

6. What it cost, and what we learned about the T2

On 1 November, a macOS 26.7.1 to 26.7.2 update was staged from System Settings at about 05:22. Work on the 27.2 build continued in a separate session, and something in that work touched the T2 while the update was pending. Which command was never recorded. Not bless, not kmutil, not nvram, not csrutil, not bputil. None of them left a log line. That is the honest state of the record.

The restart after that began a loop: a blue screen and a spinner for about three seconds, black for about thirteen, and a reset. Every Apple-signed image then failed at the same point, and the list matters:

Boot target	Result
Installed Tahoe (Apple's own sealed snapshot)	Reset
Local Recovery	Reset
A separate recovery partition, written from Apple's image, 92 of 92 chunks verified	Reset
Internet Recovery, current and original	Reset after download
Apple Diagnostics	Reset after download

Arch Linux booted normally throughout, from the same SSD, through the same T2.

Read from Linux, NVRAM held no panic record, no leftover boot arguments, and no altered security setting, and the firmware's own failure-reason field read zero: it believed its handoff to Apple's boot loader was clean. Verbose boot was switched on and confirmed, and no text ever appeared. The reset came before Apple's loader printed a line.

There was no second Mac prepared in advance. One was built during the recovery: a 2017 MacBook Pro got a Ventura partition and Apple Configurator 2.16. From it, two DFU revives completed, and neither changed anything. A DFU restore, which erases the internal storage and the T2's stored state, fixed it completely. The next boot reached Apple's recoveryOS, passed Activation Lock, and reinstalled Tahoe.

That sequence supports one inference, labelled as one. A revive replaces the T2's firmware and keeps its stored state. A restore wipes that state. The fault persisted across every Apple image and two revives, and cleared only with the restore. So it sat in persistent T2 state that a revive preserves. Which command put it there remains unknown.

It also rules three things out. Not the SSD: Linux used it throughout, and it reports healthy. Not the hardware: the restore fixed it. Not the sealed system: the snapshot was Apple's own, and after reinstall the seal is intact.

What the restore erased: the Tahoe data volume, which FileVault kept out of reach from Linux, the Arch install, the 27.2 work partition, and the recovery partition. I accepted that before the restore. The Linux firmware and configuration were saved first.

Mistakes, on the record. The project's: a boot-modifying change during a staged update, unrecorded. Claude's: a first diagnosis of a custom unsealed snapshot that forensics disproved; a wrong startup key combination early on; a call of "hardware" after the restore, retracted when the next boot reached recovery; a warning about Startup Security that came after the restore instead of before it; and a USB-writer script whose safety check looked for a field this macOS no longer prints. Grok's report: it called the revives failed when they completed, described the loop as a white spinner when it was blue, and said no second machine existed when one was built.

Rules for anyone trying this on a T2 Mac.

1. Prepare a second Mac with Apple Configurator before any boot experiment, and confirm it can see your machine in DFU mode.
2. Never begin boot-modifying work while a macOS update is staged. Install it or cancel it first.
3. Know the difference between revive and restore. A revive can complete and fix nothing. A restore erases everything.
4. Save anything Linux needs that came from macOS, especially Wi-Fi and Bluetooth firmware, before a restore.
5. Write down every boot-modifying command before you run it. The one thing this project cannot answer is which one ran.

---

7. To the people who will laugh

Some will say we just could not figure it out. In the narrow sense they are right: nothing booted, and I am not offended. But "could not figure it out" and "found the exact wall and documented why it holds" are different things, and the second one is in this paper with hashes. The commentators said there was no Intel kernel. There is one. The best-resourced team in this space has not done it and is shrinking. The map is in section 4. Humble about the outcome, firm about the evidence.

And one contrast that is the whole credibility of this paper. Early in the project a mock-up screenshot showed "macOS 27.2 Golden Gate" on this machine. It was made with AI, and it was never real. What is on the machine now says, in its own About panel, that it is a label and not a version. That is the difference between the project we started and the one we are finishing.

Appendix: evidence

Everything below is a size, a hash, a symbol count, or a log line. No Apple binaries are held in the repository.

Claim	Proof	Strength
27.2 ships a complete Intel kernel	21,128,776 B · sha256 `3c9317518535791503970fbec322b9bba70e91f3a233536205ccd77b047ffc83` · `xnu-13432.41.4~25/RELEASE_X86_64`	measured
Most of 27.2's userspace is still Intel	4,016 universal · 59 x86_64-only · 1,163 arm64-only	measured
The boot path links against 27.2's libraries	3,584 imports, 0 required misses; Tahoe control on the same checker	measured
Both GPU userspace stacks link	3,781 imports, 0 required misses	measured
WindowServer and loginwindow link	0 required misses each · Finder 3 · Dock 4	measured
Only two driver symbols in 12,887 fail	one shimmed, one a weak import Tahoe itself leaves unresolved	measured
A kernel collection can be built without root	25 entries · 26,132,480 B · sha256 `a1179078…0d408abc` · exit 0	measured
All 33 Tahoe drivers can be recovered from the live system	33 recovered, 0 missing, all x86_64	measured
A carved driver cannot enter a 27.2 collection	*Has load address* on every candidate, dependencies satisfied	measured
The missing data cannot	244 of 335 pointer sites in the first candidate have no recoverable target	measured

be reconstructed		
The loop was caused by T2-persistent state from a change during an update	every Apple image failed; two revives changed nothing; a restore fixed it; no command log exists	inferred
A mixed collection would have booted	never attempted to completion	unknown

Machine: MacBookPro16,1 · Intel Core i7-9750H · 64 GB · Apple T2.
Host OS now: macOS 26.7.2 (25G332), SIP enabled, Authenticated Root enabled, system snapshot sealed.
Target: macOS 27.2 (26B123).

The full technical report, with the build tooling and the measurements behind this paper, stays private. This appendix carries the evidence that can be shown.

Prepared 4 November 2026. Every figure here comes from a file that was measured, a log that was read, or a video that was watched. Where the record is incomplete, this paper says so instead of filling the gap.